Université Tahri Mohamed de Béchar
Faculté des sciences exactes
Département des Mathématiques et Informatique
Deuxième Année Ingénieur Informatique
Matière : Algorithmiques et complexité

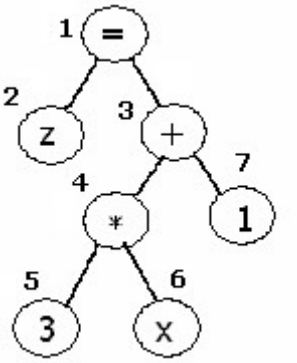
Fiche de TD n° 03
Arbres

Exercice n°1 : Donner la représentation infixée des expressions suivantes :

1- Y=(x+y)/6+z
2- $y = 6tg(3x+1) + \frac{\alpha}{\sqrt{x}}$

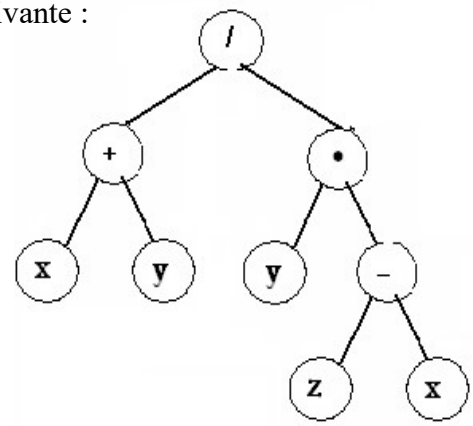
Exercice n° 02 :

Donner la représentation postfixée de l'arbre binaire suivant :



Exercice n°03 :

Soit l'expression arithmétique suivante :
E=((x+y)/(y*(z-x)))
Et sa représentation schématique est :



Donner les représentations préfixée, postfixée et infixée de E

Exercice n° 04 :

Dessiner l'arbre binaire de recherche obtenu en insérant les éléments de la liste suivante dans leur ordre d'arrivée: 6, 78, 55, 89, 15, 24, 125, 3, 11

Exercice n° 05 :

Supposons que la séquence suivante énumère les nœuds d'un arbre binaire T dans l'ordre préfixe et dans l'ordre infixé, respectivement : Préfixe : G B Q A C K F P D E R H Infixe : Q B K C F A G P E D H R Dessinez le diagramme de l'arbre.

Exercice n° 06 :

Supposons que les huit nombres suivants soient insérés dans l'ordre dans un arbre binaire de recherche T initialement vide : 50, 33, 44, 22, 77, 35, 60, 40. Dessinez l'arbre T.

Exercice n°07 :

Écrivez une fonction qui compte le nombre de nœuds dans un arbre binaire suivant les trois types de parcours (Préfixe, infixe et postfixe)

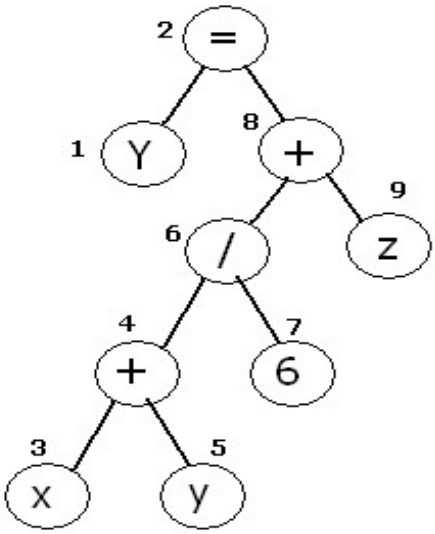
Exercice n° 08 :

Dessinez l'expression binaire pour l'expression suivante :
\(E = (3x - y)(5a + b)4\)

Solution

Exercice n° 01 :

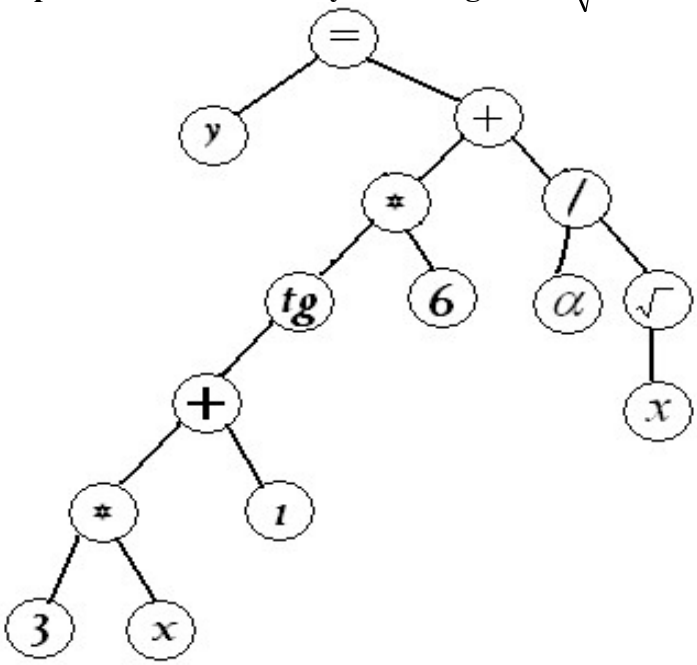
1- **L'arbre de l'expression :** Y=(x+y)/6+z



sa représentation infixée de l'expression Y=(x+y)/6+z est : **Y=x+y/6+z**
Donc on remarque qu'elle est déjà en infixe.
On n'a pas besoin de la modifier. On peut simplement évaluer la valeur de Y en remplaçant les variables x, y et z par des nombres et en effectuant les opérations selon l'ordre suivant : addition, division et addition.

Par exemple, si x = 2, y = 4 et z = 3, alors Y = (2 + 4) / 6 + 3 = 1 + 3 =

2- **Représentation infixée : y=3*x+1 tg *6+$\alpha/\sqrt{x}$**



Exercice n°02 :

Voici la représentation postfixée de l'arbre binaire :

1 **-(z)(+)=**
2- z(*)(1)+=
3- z(3)(x)*1+=

Représentation post fixée : z(3)(x)*1+=

<u>Exercice n°03 :</u>

On a, par exemple, pour l'expression E ci-dessus :
Forme infixée : x+y/y*z-x
Forme préfixée : /+xy*y-zx
Forme postfixée : xy+yzx-*/
On peut obtenir ces formes de manière récursive

Forme infixée:	**Forme préfixée:**	**Forme postfixée**:
ordre GRD	ordre RGD	ordre GDR
(+)/(*)	/(+)(*)	(+)(*)/
(x)+(y)/(y)*(-)	/+(x)(y)*(y)(-)	(x)(y)+(y)(-)*/
x+y/y*(z)-(x)	/+xy*y-(z)(x)	xy+y(z)(x)-*/
x+y/y*z-x	/+xy*y-zx	xy+yzx-*/

<u>Exercice n° 04 :</u>

```
        6
       / \
      3   78
         /  \
        55   89
       /       \
    15          125
   /   \
  15   24
```

Cela représente l'arbre binaire de recherche après avoir inséré tous les éléments de la liste donnée dans l'ordre d'arrivée.

<u>Exercice n° 05</u>
Préfixe : G B Q A C K F P D E R H
Infixe : Q B K C F A G P E D H R

Voici les étapes permettant de dessiner le diagramme de l'arbre binaire à partir des séquences préfixe et infixe données:

1. **Racine de l'arbre :** La racine est G, le premier nœud du parcours préfixe.

2. **Sous-arbres de G :**
   - À gauche de G dans le parcours infixé : B Q A C K F.
   - À droite de G dans le parcours infixé : P E D H R.

3. **Sous-arbre gauche de G :**
   - Racine : B (premier nœud à gauche de G dans le parcours infixé).
   - Sous-arbre gauche de B : Q.
   - Sous-arbre droit de B : A C K F.
     - Sous-arbre gauche de A : C K F.
       - Sous-arbre gauche de C : K.
       - Sous-arbre droit de C : F.
     - Sous-arbre droit de A : N/A (A n'a pas de sous-arbre droit).
   - Sous-arbre gauche de Q : N/A (Q n'a pas de sous-arbre gauche).
   - Sous-arbre droit de Q : N/A (Q n'a pas de sous-arbre droit).

4. **Sous-arbre droit de G :**
   - Racine : P (premier nœud à droite de G dans le parcours infixé).
   - Sous-arbre gauche de P : N/A (P n'a pas de sous-arbre gauche).
   - Sous-arbre droit de P : D E R H.
     - Sous-arbre gauche de D : E.
     - Sous-arbre droit de D : R H.
       - Sous-arbre gauche de R : H.
       - Sous-arbre droit de R : N/A (R n'a pas de sous-arbre droit).
     - Sous-arbre gauche de E : N/A (E n'a pas de sous-arbre gauche).
     - Sous-arbre droit de E : N/A (E n'a pas de sous-arbre droit).
     - Sous-arbre gauche de H : N/A (H n'a pas de sous-arbre gauche).
     - Sous-arbre droit de H : N/A (H n'a pas de sous-arbre droit).

5. **Arbre complet :**

```
        G
       / \
      B   P
     / \   \
    Q A     D
       / \   / \
      C K   E   R
         \
          F
           \
            H
```

Cela représente l'arbre binaire complet construit selon les étapes détaillées, incluant les nœuds K et F.
Voici un autre diagramme correspondant à l'arbre binaire T décrit par les séquences préfixe et infixe données :

```
      G
     / \
    /   \
   /     \
  B       P
 / \       \
/   \       \
Q    A       D
    / \     / \
   /   \   /   \
  C   F E     R
 / \         /
/   \       /
K    F      H
```

Assurez-vous de vérifier que l'ordre préfixe de l'arbre obtenu correspond à la séquence donnée, de même pour l'ordre infixe .

Exercice n° 06

Pour construire l'arbre binaire de recherche (ABR) T, où les éléments sont insérés dans l'ordre, suivez ces étapes :

1. **Insérer 50 :** Le premier élément, 50, devient la racine.
2. **Insérer 33 :** 33 est inférieur à 50, donc il va à gauche de 50.
3. **Insérer 44 :** 44 est supérieur à 33 mais inférieur à 50, donc il va à droite de 33.
4. **Insérer 22 :** 22 est inférieur à 33, donc il va à gauche de 33.
5. **Insérer 77 :** 77 est supérieur à 50, donc il va à droite de 50.
6. **Insérer 35 :** 35 est supérieur à 33 mais inférieur à 44, donc il va à droite de 33.
7. **Insérer 60 :** 60 est supérieur à 50 mais inférieur à 77, donc il va à gauche de 77.
8. **Insérer 40 :** 40 est supérieur à 33 mais inférieur à 44, donc il va à gauche de 44.

Le résultat est l'arbre binaire de recherche T suivant :

```
50
/   \
```

```
      /       \
     33       77
    /  \       /
  22   44    60
       /
     35
       \
        40
```

Exercice n°07 :

Voici comment vous pouvez adapter la fonction pour les trois types de parcours (Préfixe, infixe et postfixe)

1- Parcours Préfixe

```
class Noeud:
    def __init__(self, valeur):
        self.valeur = valeur
        self.gauche = None
        self.droit = None

def parcours_prefixe(arbre):
    if arbre is not None:
        print(arbre.valeur, end=" ")
        parcours_prefixe(arbre.gauche)
        parcours_prefixe(arbre.droit)

# Exemple d'utilisation:
# Création d'un arbre binaire simple
racine = Noeud(1)
racine.gauche = Noeud(2)
racine.droit = Noeud(3)
racine.gauche.gauche = Noeud(4)
racine.gauche.droit = Noeud(5)
#       1
#      / \
#     2   3
#    / \
#   4   5
# Appel de la fonction pour le parcours en préfixe
print("Parcours en préfixe:")
parcours_prefixe(racine)
```

2- Parcours Infixe

```
def parcours_infixe(arbre):
    if arbre is not None:
        parcours_infixe(arbre.gauche)
```

```
        print(arbre.valeur, end=" ")
        parcours_infixe(arbre.droit)

# Exemple d'utilisation:
# Appel de la fonction pour le parcours en infixe
print("Parcours en infixe:")
parcours_infixe(racine)
```

3- Parcours Postfixe

```
def parcours_postfixe(arbre):
    if arbre is not None:
        parcours_postfixe(arbre.gauche)
        parcours_postfixe(arbre.droit)
        print(arbre.valeur, end=" ")

# Exemple d'utilisation:
# Appel de la fonction pour le parcours en postfixe
print("Parcours en postfixe:")
parcours_postfixe(racine)
```

Chaque fonction modifie l'ordre dans lequel elle effectue l'impression du nœud par rapport à ses appels récursifs, conformément à la méthode de parcours correspondante.